Сейчас наша архитектурап содержит 2 небольшие проблемы которые нам нужно исправть.

```
 /// </summary>
 public static class NnClLoc
 {
     /// <summary>
     /// Выполняет динамическую сборку и запуск сплавленного OpenCL-ядра на
основе C# кода.
     /// </summary>
     /// <param name="expression">Дерево выражений математического
блока.</param>
     public static void Run(Expression<Action> expression)
     {
         if (expression == null) throw new
ArgumentNullException(nameof(expression));

         // 1. Создаем визитор и запускаем обход дерева выражений C#
         var visitor = new LocalExpressionVisitor();
         visitor.Visit(expression.Body);

         // Если в блоке кода не было обнаружено тензоров, выходим
         if (visitor.UsedTensors.Count == 0) return;

         // 2. ВАЖНОЕ ВЫЧИСЛЕНИЕ: Находим максимальный размер среди всех
задействованных тензоров
         int maxTensorSize = visitor.UsedTensors.Values.Max(v => v.Tensor.Size);

         // 3. Генерируем финальный Си-текст OpenCL ядра
         string generatedCode = visitor.GenerateKernelSource();

         // Получаем глобальный контекст и текущую активную очередь через наш
реестр
         CommandQueue queue = DeviceQueueRegistry.GetCurrent();
         Context context = queue.Context;
         Device device = queue.Device;

         // 4. Компилируем ядро «на лету» (или берем готовое из JIT-кэша)
         Program program = JITProgramCache.GetOrCompile(context, device,
generatedCode);
         using Kernel kernel = program.CreateKernel("fused_elementwise_kernel");

         // 5. Динамически привязываем аргументы к скомпилированному ядру
         int argIndex = 0;

         // Сначала передаем пары: [Буфер VRAM, Длина тензора]
         foreach (var kvp in visitor.UsedTensors)
         {
             LocalVariable variable = kvp.Value;

             kernel.Arguments[argIndex++].SetValue(variable.Tensor.DataBuffer);
             kernel.Arguments[argIndex++].SetValue(variable.Tensor.Size);
         }

         // Затем передаем захваченные внешние скаляры (например, float q)
         foreach (var kvp in visitor.CapturedScalars)
         {
             float scalarValue = kvp.Value;
             kernel.Arguments[argIndex++].SetValue(scalarValue);
         }

         // 6. Запускаем сплавленное ядро на GPU
         // Размер сетки потоков равен размеру наибольшего вектора
         IntPtr globalSize = new IntPtr(maxTensorSize);
```

```
            // Отправляем всю пачку математики на видеокарту ОДНИМ пинком CPU
            queue.EnqueueNDRangeKernel(kernel,  new[] { globalSize }, null);

            // Жесткая синхронизация: гарантирует, что GPU завершил вычисления
            // до того, как CPU выйдет из блока NnClLoc.Run
            queue.Finish();
        }
    }
```

1. NopenCL может передавть как аргументы либо целые числа, либо адреса памяти (буферы). В тоже время поскольку ядро компуеться когда q уже известно, нам рациональнее заменить q на его значение (что ожет допонительно ускорить код).

```
using System;
using System.Collections.Generic;
using System.Linq.Expressions;
using System.Text;

namespace cstorchcl.compiler
{
    /// <summary>
    /// Обходит C# дерево выражений и транслирует его в оптимизированный OpenCL
C код.
    /// </summary>
    public class LocalExpressionVisitor : ExpressionVisitor
    {
        private readonly StringBuilder _bodyCode = new StringBuilder();

        // Списки для отслеживания используемых тензоров и захваченных скалярных
констант (q)
        public Dictionary<string, LocalVariable> UsedTensors { get; } = new
Dictionary<string, LocalVariable>();
        public Dictionary<string, float> CapturedScalars { get; } = new
Dictionary<string, float>();

        /// <summary>
        /// Генерирует полный исходный код OpenCL ядра на основе разобранного
выражения.
        /// </summary>
        public string GenerateKernelSource()
        {
            var source = new StringBuilder();

            // 1. Формируем сигнатуру ядра динамически на основе обнаруженных
аргументов
            source.Append("kernel void fused_elementwise_kernel(\n");

            // Добавляем буферы глобальной памяти для тензоров
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    global float* {kvp.Key}, const int
{kvp.Key}_len,\n");
            }

            // Добавляем захваченные скаляры (например, float q)
            int scalarIdx = 0;
            foreach (var kvp in CapturedScalars)
            {
                source.Append($"    const float {kvp.Key}");
                if (scalarIdx < CapturedScalars.Count - 1 || false)
source.Append(",\n");
```

```
                scalarIdx++;
            }

            // Убираем возможную лишнюю запятую в конце сигнатуры и закрываем
скобку
            if (source.ToString().EndsWith(",\n")) source.Length -= 2;
            source.Append(") {\n");
            source.Append("    int tid = get_global_id(0);\n\n");

            // 2. Фаза Оптимизации GPU: Объявление private переменных
(регистров) и безопасное чтение из VRAM
            source.Append("    // Чтение во внутренние регистры (Оптимизация
private-памяти)\n");
            foreach (var kvp in UsedTensors)
            {
                source.Append($"    float priv_{kvp.Key} = {kvp.Key}[min(tid,
{kvp.Key}_len - 1)];\n");
            }
            source.Append("\n");

            // 3. Добавляем тело вычислений, которое мы распарсили из C# кода
            source.Append("    // Вычисления над регистрами\n");
            source.Append(_bodyCode.ToString());
            source.Append("\n");

            // 4. Финальная фаза: Запись результатов из регистров обратно во
VRAM с проверкой границ
            source.Append("    // Безопасное сохранение результатов во VRAM\n");
            foreach (var kvp in UsedTensors)
            {
                // Для простоты примера считаем, что все тензоры, участвующие в
левой части выражений, перезаписываются
                source.Append($"    if (tid < {kvp.Key}_len) {{ {kvp.Key}[tid] =
priv_{kvp.Key}; }}\n");
            }
            source.Append("}");

            return source.ToString();
        }

        // Перехватываем математические операции присваивания или бинарные
операции (/, +, *, -=)
        protected override Expression VisitBinary(BinaryExpression node)
        {
            // Определяем левую переменную (куда пишем)
            if (node.Left is MemberExpression leftMember && leftMember.Type ==
typeof(LocalVariable))
            {
                string varName = leftMember.Member.Name;

                // Переводим операцию C# в операцию Си
                string opStr = node.NodeType switch
                {
                    ExpressionType.AddAssign or ExpressionType.Add => "+=",
                    ExpressionType.SubtractAssign or ExpressionType.Subtract =>
"-=",
                    ExpressionType.MultiplyAssign or ExpressionType.Multiply =>
"*=",
                    ExpressionType.DivideAssign or ExpressionType.Divide =>
"/=",
                    _ => "="
                };

                _bodyCode.Append($"    priv_{varName} {opStr} ");
```

```
                // Разбираем правую часть выражения (это может быть константа,
захваченная переменная или функция)
                Visit(node.Right);
                _bodyCode.Append(";\n");

                return node;
            }

            return base.VisitBinary(node);
        }

        // Перехватываем вызовы математических функций вроде nnclloc.sin(A)
        protected override Expression VisitMethodCall(MethodCallExpression node)
        {
            if (node.Method.DeclaringType == typeof(nnclloc))
            {
                string funcName = node.Method.Name.ToLower(); // sin, cos
                _bodyCode.Append($"{funcName}(");

                // Разбираем аргумент внутри функции (например, priv_A)
                if (node.Arguments[0] is MemberExpression argMember)
                {
                    _bodyCode.Append($"priv_{argMember.Member.Name}");
                }

                _bodyCode.Append(")");
                return node;
            }

            return base.VisitMethodCall(node);
        }

        // Перехватываем чтение внешних переменных типа "q", захваченных
контекстом C# (замыкания)
        protected override Expression VisitMember(MemberExpression node)
        {
            // Если это наш объект LocalVariable, регистрируем его
            if (node.Type == typeof(LocalVariable))
            {
                string name = node.Member.Name;
                if (!UsedTensors.ContainsKey(name))
                {
                    // Извлекаем объект LocalVariable из контекста выполнения C#
                    var objectMember = Expression.Convert(node, typeof(object));
                    var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
                    var localBuffer = (LocalVariable)getterLambda.Compile()();

                    UsedTensors.Add(name, localBuffer);
                }
                return node;
            }

            // Если это захваченный внешний примитив (float/double), например
переменная "q"
            if (node.Type == typeof(float) || node.Type == typeof(double))
            {
                string name = node.Member.Name;
                if (!CapturedScalars.ContainsKey(name))
                {
                    var objectMember = Expression.Convert(node, typeof(object));
                    var getterLambda =
Expression.Lambda<Func<object>>(objectMember);
```

```
                    float val = Convert.ToSingle(getterLambda.Compile()());

                    CapturedScalars.Add(name, val);
                }
                _bodyCode.Append(name);
                return node;
            }

            return base.VisitMember(node);
        }

        // Перехватываем прямые константы в коде, например "0.5" в выражении B
+= 0.5
        protected override Expression VisitConstant(ConstantExpression node)
        {
            if (node.Value is float f)
            {

_bodyCode.Append(f.ToString(System.Globalization.CultureInfo.InvariantCulture) +
"f");
            }
            else if (node.Value is double d)
            {

_bodyCode.Append(d.ToString(System.Globalization.CultureInfo.InvariantCulture) +
"f");
            }
            else
            {
                base.VisitConstant(node);
            }
            return node;
        }
    }
}
```

```
38
39      /// <summary>
40      /// Статические математические функции для локального конвейера (аналог Math).
41      /// </summary>
        Ссылок: 1
42      public static class nnclloc
43      {
            Ссылок: 0
44          public static LocalVariable sin(LocalVariable v) => v;
            Ссылок: 0
45          public static LocalVariable cos(LocalVariable v) => v;
            Ссылок: 0
46          public static LocalVariable tangent(LocalVariable v) => v;
47      }
48  }
49
```

Зденсь 2 момента:
1. По идее на tangent a tanh?
2.Нужно реализовать функцию drout которая будет занулять на gpu ,это весьма важная функция для обучения сетей.

В старой версии был такой вариант:

```
// Быстрый генератор псевдослучайных чисел на основе индекса потока и seed
inline float nn_rand(int tid, unsigned int seed) {
    unsigned int x = tid + seed;
    x = (x ^ 61) ^ (x >> 16);
    x += (x << 3);
    x = x ^ (x >> 4);
    x *= 0x27d4eb2d;
    x = x ^ (x >> 15);
    // Возвращает float в диапазоне [0.0, 1.0]
    return (float)x / 4294967296.0f;
}

// Функция инлайн-дропаута для прямого прохода
inline float nn_dropout(float x, float p, int tid, unsigned int seed) {
    if (p <= 0.0f) return x;
    if (p >= 1.0f) return 0.0f;

    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return x / (1.0f - p); // Масштабирование Inverted Dropout
    }
}

// Функция инлайн-дропаута для обратного прохода
inline float nn_dropout_backward(float grad_out, float p, int tid, unsigned int
seed) {
    float r = nn_rand(tid, seed);
    if (r < p) {
        return 0.0f;
    } else {
        return grad_out / (1.0f - p);
    }
}
```